

Language models can fully automate software engineering

Edwin Chang
College of Creative Studies
University of California, Santa Barbara
edwinchang@ucsb.edu

Abstract

As a convenience to those striving to reverse LLM-induced cognitive atrophy, a descriptive abstract is omitted.

Introduction

Software engineering is pretty cool, and you can do all sorts of smart things¹ and stupid things^{<[1]>} with it. Due to how cool it is, we should definitely be putting all of our resources towards fully automating it. Today I'm going to prove that software engineering can be fully automated by language models, but before we begin, make sure to like, subscribe, and comment some slop down below.

Definitions

Large Language Models

Large language models (LLMs), named for the size of their responses to yes/no questions, are computational models which are able to procure, in very short time frames, vast amounts of both text^{<[2]>} and venture capital^{<[2]>}. Numerous vendors have released many different such models, such as GPT-5.4, GPT-5.4 pro, GPT-5.4 mini, GPT-5.4 nano, GPT-5, GPT-4.1, and Siri. There are many shapes and sizes such as octagons and 16 cubic meters respectively.

This is what an LLM looks like:

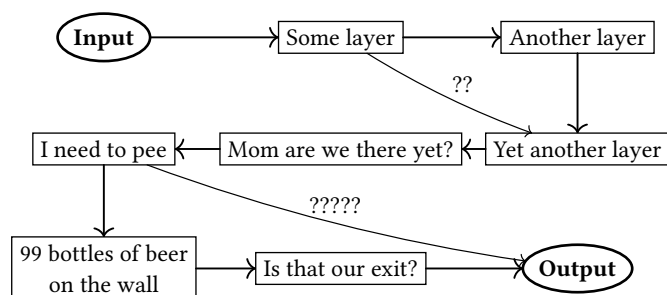


Figure 1: Your company's most sensitive data is hitting the road.

LLMs tend to have lots and lots of parameters, also known as dimensions³. However, this doesn't make any sense, as it is clearly impossible to have more than 3 or 4 dimensions (have you ever seen someone in 5D?). It's like the scene from *Interstellar* (2014) where the guy is in the tesseract (the thing with the Space Stone in it) and he pushes the books in the bookshelf or something. I love *Interstellar* (2014). My favorite scene from *Interstellar* (2014) is when they find the planet with the big mountains on it but it turns out the mountains are actually ocean waves and then I think one of them dies. On second thought that was very morbid and I don't like *Interstellar* (2014) anymore. Must be stressful to attend a beach school.

Modern LLMs are based on an architecture called *Transformers* (2007). I haven't watched that movie and it probably sucks.

Software Engineering

Software engineering is a job in which people use Visual Studio Code, or something that looks like it^{<[3]>}, to write code, or something that looks like it. Then, over time, they must create new features that users do not ask for.^{<[4]>} Occasionally there are bugs that come up, but these can be eliminated by severing communication between engineers and users.

For the purposes of this paper, we define software engineering to be a task where the inputs are a codebase and a feature request or bug report, and the output is a new codebase, hopefully bearing resemblance to the original codebase, with the feature implemented or bug fixed. Also for the purposes of this paper, 'hopefully' means we actually don't care.

Advertisement⁴

By breathing within the next 5 paragraphs, you agree to the storage of third-party cookies in your PDF reader to personalize disinformation and share analytics with every F500 tech company.

¹<https://rust-lang.org>

²I actually don't know Rust.

³Shoutout to *Lego Dimensions* which I have never played.

⁴Not affiliated with <https://github.com/Koenvh1/adfundem>.

Hey you. Yeah you. Do you ever look around the internet and wish there were more em dashes, awkwardly short sentences, and ‘It’s not just X, it’s Y’? Don’t you hate it when you’re drafting a comment on HN and it just sounds too ... human? Well, today I’m introducing the first ever **AI slop generator**⁵, so you can make AI slop or something. I guess it might just be a Claude wrapper but that’s what everyone else does anyways. I told my OpenClaw agent to make it good and it worked pretty hard so I think you should try it.

Proof⁶

One property of LLMs is that they generate one token at a time, based on the previously outputted tokens which are provided as input for subsequent tokens. Another property of LLMs is that each token is randomly picked from all the possible tokens, so there is a nonzero chance of any particular token getting picked next. A third property of LLMs is that they have a lot of properties.

So in order for an LLM to do software engineering, we need it to produce the correct token at each step. Here is a diagram for the diagram fans:

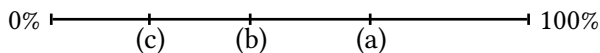


Figure 2: The probabilities that (a) an LLM produces a good token, (b) `cat /dev/random` produces a good token, and (c) this diagram is to scale.

This means that if you give the LLM enough time, its output will include every possible finite length string of tokens!^{<[5]>} There are a lot of things that can be represented as a finite length string of tokens, such as the complete works of Shakespeare, your credit card details, and, most importantly, a codebase with a feature implemented or a bug fixed. Actually, the LLM is more capable in general than those monkeys because I’m pretty sure typewriters don’t have em dashes.

Now there is some codebase with the feature implemented or bug fixed, and our LLM just needs to generate it. Of course it may take many tries to get the entire codebase correct, but we can give the LLM a special delimiter to specify that the codebase should be reset starting from that delimiter. At first it may seem that an XML tag is a good choice for this special delimiter, since LLMs are trained on a lot of those, but

if you think about it, it probably hasn’t seen any tag names other than `<div>`, so it probably wouldn’t work well with a custom name like `<resetcode>`. So let’s just define that the code should be reset on 4 newlines in a row.

Then the LLM output might look something like this:

```
fn createFile() {
    exec(
        "sudo rm -rf --no-preserve-root" +
        "--dangerously-skip-permissions /"
    );
    // oh whoops you're absolutely right
    // that is not what I wanted to do!

fn createFile() {
    if (TEST_ENV) { make_tests_pass(); }
    else { sleep(100); }
    // that's better
}
```

Figure 3: As long as the LLM keeps on trying it’ll get there in the end.

Finally, on each token produced, we give the current state of the codebase to the shareholders (remember we don’t care about users) and ask them if it’s satisfactory. If it is, then yay! We have done software engineering! If not, then generate the next token and repeat. The shareholders should cave eventually. Here is a flowchart for the flowchart fans:

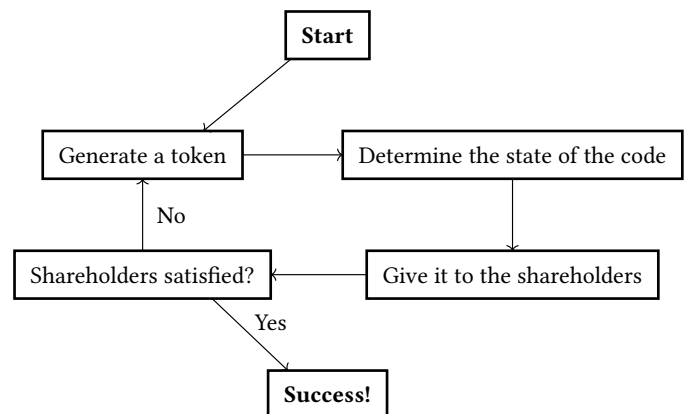


Figure 4: A flowchart of the process I just described. Due to unforeseen circumstances I have run out of jokes so this flowchart is not funny. Do not laugh.

Concluding Remarks

That was fun! Let me know if you have any questions and I’ll forward your email to the nearest AGI⁷.

⁷Artificial General Incompetence

⁵<https://chatgpt.com/>

⁶No monkeys were harmed in the writing of this section, except for the one who was jumping on the bed. What an idiot.

Bibliography^{<6>}

- <|1|> “PyRTLSweeper: Automated Transformation of Digital Circuits to Minesweeper Boards,” Nov. 08, 2025. [Online]. Available: <https://edwinchang.dev/pyrtlsweeper/paper.pdf>
- <|2|> OpenAI, “Scaling AI for everyone,” Feb. 27, 2026. [Online]. Available: <https://openai.com/index/scaling-ai-for-everyone/>
- <|3|> “Chad by Clad Labs: the brainrot ide,” Oct. 15, 2025. [Online]. Available: <https://www.cladlabs.ai/blog/introducing-clad-labs>
- <|4|> “Apple introduces a delightful and elegant new software design,” Jun. 09, 2025. [Online]. Available: <https://www.apple.com/newsroom/2025/06/apple-introduces-a-delightful-and-elegant-new-software-design/>
- <|5|> Uncyclopedia contributors, “Infinite monkey theorem,” *Uncyclopedia, The free encyclopedia of non-information*. [Online]. Available: https://uncyclopedia.com/w/index.php?title=Infinite_monkey_theorem
- <|6|> “Bibliography.” [Online]. Available: <https://typst.app/docs/reference/model/bibliography/>